

2-1 Introduction

C++ provides a special member function called the constructor which enables an object to initialize itself when it is created. This is known as automatic initialization of objects. It also provides another member function called the destructor that destroys the objects when they are no longer required.

2-2 constructors

- ✚ A constructor is a “special” member function whose task is to initialize the objects of its class. A constructor is a member function that is executed automatically whenever an object is created.
- ✚ It is special because its name is the same as the class name. The constructor is invoked whenever an object of its associated class is created. It is called constructor because it constructs the values of data members of the class.

A constructor is declared and defined as follows:

```
// class with a constructor

class integer {

int m , n;

public:

integer ( void );           // constructor declared

..... };

integer:: integer (void )   // constructor defined

{
m=0; n=0;
}
```

When a class contains a constructor like the one defined above, it is guaranteed that an object created by the class will be initialized automatically. For example, the declaration **integer int1;** //object int1 created.

The constructor functions have some special characteristics:

- ❖ They should be declared in the public section.
- ❖ They are invoked automatically when the objects are created.
- ❖ They do not have return types, not even void and therefore, they cannot return values.

- ❖ They cannot be inherited, though a derived class can call the base class constructor.
- ❖ Like other C++ functions, they can have default arguments.
- ❖ We can be defined as **inline** function

4.3 Parameterized Constructors

The constructor **integer ()**, defined above, initializes the data members of all the objects to zero. However, in practice it may be necessary the various data elements of different objects with different values when they are created. C++ permits us to achieve this objective by passing arguments to the constructor function when the objects are created.

```
class integer
{
  int m , n;
public:
  integer ( int x , int y );           // parameterized constructor
  .....
  .....
};
integer:: integer (int x , int y )    // constructor defined
{
  m = x; n = y;
}
```

In above program we must pass the initial values as arguments to the constructor function when an object is declared.

This can be done in two ways:

- By calling the constructor **explicitly**.
- By calling the constructor **implicitly**.

The following declaration illustrates the first method:

```
integer int1= integer(0,150);           //explicit call
```

This statement creates an **integer** object **int1** and passes the values 0 and 150 to it.

The second is implemented as follows:

```
integer int1(0,150); //implicit call (shorthand)
```

2-4 Default values

You can set default values for class members in C++ using two primary methods: **in-class member initializers** (C++11 and later) or by using a **default constructor** (a constructor with no parameters or all parameters having default values).

Method 1: In-Class Member Initializers (Recommended)

This is the most straightforward and modern C++ approach. You assign default values directly in the class definition. The compiler automatically uses these values for any object created without an explicit constructor overriding them.

Example

```
#include <iostream>
using namespace std;
class Car {
private:
    string make = "Unknown Make";           // Default value using in-class initializer
    string model = "Unknown Model";         // Default value using in-class initializer
    int year = 0;

public:
    // Default constructor (can be explicitly defaulted if needed, but not strictly necessary here)
    Car() = default;

    // Parameterized constructor to set specific values
    Car(string make_val, string model_val, int year_val)
        : make(make_val), model(model_val), year(year_val) {}

    // Member function to display car info
    void displayCarInfo() {
        cout << "Make: " << make << ", Model: " << model << ", Year: " << year << endl;
    }
};

int main() {
    // Object created using the default values
    Car defaultCar;
    cout << "Default Car Info: ";
    defaultCar.displayCarInfo();

    // Object created with specific values using the parameterized constructor
    Car customCar("Toyota", "Camry", 2023);
    cout << "Custom Car Info: ";
```

هنا لم نكتب body للدالة لان المترجم يأخذ القيم الافتراضية للكلاس

يهيء المتغيرات مباشرة بالقيم التي تعطى عند انشاء الكائن.

```
customCar.displayCarInfo();  
  
return 0;  
}
```

Output:
Default Car Info: Make: Unknown Make, Model: Unknown Model,
Year: 0
Custom Car Info: Make: Toyota, Model: Camry, Year: 2023

Method 2: Using a Constructor with Default Arguments

You can also define a single constructor where all parameters have default values. This constructor acts as both a default constructor (when no arguments are provided) and a parameterized constructor (when arguments are provided).

Example

```
#include <iostream>  
#include <string>  
  
class Car {  
private:  
    std::string make;  
    std::string model;  
  
public:  
    // Constructor with default arguments for make and model  
    Car(std::string make_val = "Unknown Make", std::string model_val =  
"Unknown Model")  
        : make(make_val), model(model_val) {  
        // Additional setup can go here  
    }  
  
    // Member function to display car info  
    void displayCarInfo() {  
        std::cout << "Make: " << make << ", Model: " << model << std::endl;  
    }  
};  
  
int main() {  
    // Calls the constructor using default values  
    Car defaultCar;  
    std::cout << "Default Car Info: ";  
    defaultCar.displayCarInfo();  
  
    // Calls the constructor with specific values  
    Car customCar("Honda", "Civic");  
    std::cout << "Custom Car Info: ";  
    customCar.displayCarInfo();  
  
    return 0;  
}
```

Output:
Default Car Info: Make: Unknown Make, Model: Unknown
Model
Custom Car Info: Make: Honda, Model: Civic

2-5 multiple constructors in one class

a C++ class can have **multiple constructors**. This is known as **constructor overloading**, a key feature of object-oriented programming that allows for flexible object creation and initialization in various ways.

Key Rules for Multiple Constructors

To have multiple constructors in a single class, they must follow the rules of function overloading, which means each constructor must have a unique **signature** (parameter list):

- **Different number of parameters.**
- **Different types of parameters.**
- **Different order of parameters.**

The compiler determines which constructor to call based on the arguments provided when an object is created.

Example

Here is a C++ example demonstrating constructor overloading in a `Box` class:

```
#include <iostream>
using namespace std;

class Box {
public:
    double length;
    double width;
    double height;

    // 1. Default constructor (no parameters)
    Box() {
        length = 1.0;
        width = 1.0;
        height = 1.0;
    }

    // 2. Constructor with one parameter (cube)
    Box(double side) {
        length = side;
        width = side;
        height = side;
    }

    // 3. Constructor with three parameters (custom box)
    Box(double l, double w, double h) {
        length = l;
        width = w;
        height = h;
    }

    // Method to display volume
```

```
void displayVolume() {  
    cout << "Volume: " << length * width * height << endl;  
}  
};  
  
int main() {  
    Box box1;           // Calls the default constructor  
    Box box2(5.0);      // Calls the constructor with one parameter  
    Box box3(2.0, 3.0, 4.0); // Calls the constructor with three parameters  
  
    box1.displayVolume();  
    box2.displayVolume();  
    box3.displayVolume();  
  
    return 0;  
}
```

Output:

Volume: 1

Volume: 125

Volume: 24

Benefits

- **Flexibility** in object initialization.
- **Cleaner code** by avoiding complex initialization logic outside the class.
- Ensures objects start in a **valid state**, regardless of how much initial data is available.

2-5 Destructor

in C++ is a special member function of a class that is automatically called when an object's lifetime ends, or it goes out of scope. Its primary purpose is to perform necessary clean-up operations, such as **releasing dynamically allocated memory, closing file handles, or freeing other system resources**, to prevent memory leaks and ensure proper termination.

Key Characteristics

- **Naming:** The destructor has the same name as the class, but is prefixed by a tilde (~) character (e.g., ~MyClass() for a class MyClass).
- **No Return Type:** A destructor does not have a return type, not even void.
- **No Arguments:** A destructor does not accept any parameters.
- **Uniqueness:** A class can have only one destructor; it cannot be overloaded.

- **Automatic Invocation:** The compiler automatically calls the destructor.
- **Default Destructor:** If a user-defined destructor is not provided, the compiler generates a default, inline, public destructor with an empty body. This is sufficient for most cases, but not when a class manages resources that require explicit deallocation.

Example

```
#include <iostream>
using namespace std;

class Example {
public:
    // Constructor
    Example() {
        cout << "Constructor called" << endl;
    }

    // Destructor
    ~Example() {
        cout << "Destructor called" << endl;
    }
};

int main() {
    Example ex; // Constructor called
    // Object 'ex' goes out of scope at the end of main
    return 0; // Destructor called automatically here
}
```